

Central Processing Unit Design 期末 專題報告

從 MIPS R2000 到 RISC-V : 5
Stage Pipeline CPU 設計

學生：黎建佑

指導教授：張孟洲 教授

中華民國 115 年 6 月

摘要

本次設計之處理器為一具備 5-Stage Pipeline 的 RISC-V (RV32I)。

設計重點在於：

1. **Control Hazard 最佳化**：將分支判斷提前至 ID Stage 執行，配合 Flush 機制，使分支延遲縮短至 1 週期。
2. **應對衍生 Data Hazard**：設計 Branch Comparator、Branch Forwarding，以及應對 Hazard 的 Stall 機制。
3. **Zicsr 擴充功能與基本 Exception 偵測**：完整實作 RISC-V 官方 Zicsr 擴充指令集，並建立控制狀態暫存器 (CSR)，針對特定 Exception 的自動偵測、跳到中斷處理程式與基本現場記錄功能，為處理器核心奠定軟硬體協同設計基礎

關鍵詞：5-Stage Pipeline、RISC-V (RV32I) 指令集、分支判斷提前、Zicsr 擴充功能

目錄

摘要	2
目錄	3
專題評分依據：修改與新增設計對照表	4
壹、設計內容	5
一、指令集	5
二、5-Stage Pipeline	5
三、Pipeline Hazards	6
四、Exception 與中斷處理機制	7
貳、設計方法	10
一、ID-Stage Early Branch Comparator	10
二、Branch Forwarding Logic	10
三、Branch Hazard	11
四、CSR 實作與 Exception 處理	12
參、模擬結果	15
一、hazard.asm	15
二、branch.asm	16
三、exception.asm	19
四、CMDtest.asm	21
肆、討論	24

專題評分依據：修改與新增設計對照表

本專題以林慶樺學長之 MIPS R2000 Pipeline 設計為藍本，全面更換為 RISC-V 架構。以下為這次期末專題之具體硬體修改與新增實作摘要：

專題指標與設計要求	學長 MIPS 原始設計	本專案 RISC-V 新增與修改實作
指令集更換	傳統 MIPS R2000 指令集。 解碼時暫存器 rd、rt 欄位浮動，需經多工器（MUX）選擇。	1. 全面更換為 RISC-V RV32I 指令集（共 39 條）。 2. 解碼路徑優化： RISC-V 暫存器欄位（rs1, rs2, rd）位置對齊，因此可以拔除 RegDst MUX 選擇器，使 ID Stage 的延遲降低。
Control Hazard 最佳化與衍生 Data Hazard 處理	僅具備 Branch Hazard 的判斷，缺少 Branch Forwarding 分支判斷較晚引發 2 週期的分支懲罰。	1. 分支判斷提前： 將分支判斷提前至 ID Stage 配合 Pipeline Flush 使分支懲罰降為 1 週期。 2. 應對衍生 Data Hazard： 為配合早期分支判斷，拉出由 EX Stage 直接前推至 ID Stage 比較器、以及 MEM Stage 前推至 ID Stage 的資料路徑。配合 Stall 機制確保資料的正確。
例外與中斷處理	Exception 發生時進行跳轉，使用 Latch 進行現場狀態保存。	1. Control and Status Register（CSR_Reg）實作： 實作標準 Machine Mode 暫存器：mepc（現場返回點）、mcause（異常原因編碼）與 mtvec（中斷處理程式位址，預設初始值為 32'h0000_0080）。 2. 實作 Zicsr 指令集： 開通共 6 條 Zicsr 原子操作指令，建立 CSR 讀寫通路。

壹、設計內容

一、指令集

本設計支援 RISC-V 的 RV32I 基礎指令集(不包含 FENCE 指令)，
加上 Zicsr 擴充指令，總計 45 條指令。

- R-Type - ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU
- I-Type - ADDI, ANDI, ORI, XORI, SLLI, SRLI, SRAI, SLTI, SLTIU, LB, LH, LW, LBU, LHU, JALR, ECALL, EBREAK, CSRRW, CSRRS, CSRRC, CSRRWI, CSRRSI, CSRRCI
- S-Type - SB, SH, SW
- B-Type - BEQ, BNE, BLT, BGE, BLTU, BGEU
- U-Type - LUI, AUIPC
- J-Type- JAL

註：本 CPU 為單核心、順序執行 5 Stage Pipeline，無架構錯序風險，故硬體將 FENCE 指令視為 NOP 氣泡安全流過。

二、5-Stage Pipeline

1. **Instruction Fetch (IF)**：依據當前 PC 值從 Instruction Memory 中存取指令，並更新 PC。

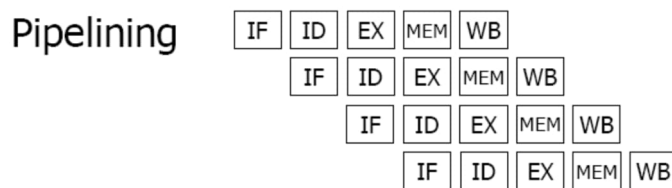
2. Instruction Decode (ID) :

- 對指令進行解碼，產生控制訊號。
- 自 Register File 讀取暫存器數值。
- 早期分支評估 (Early Branch Decision) : 在 ID 階段直接利用 Branch_Comparator 進行分支條件計算與跳轉。

3. Execute (EX) : ALU 根據控制訊號進行算術邏輯運算或地址計算。

4. Memory Access (MEM) : 對 Data Memory 進行讀寫操作 (LW、SW 等)。

5. Write Back (WB) : 將運算結果或讀出的資料寫回 Register File 的目的暫存器 (Rd)。



三、Pipeline Hazards

由於分支判斷提前至 ID Stage 執行，導致暫存器的讀取時

機比傳統 EX 判斷更早，更容易產生 Hazard：

1. Branch Forwarding：當分支指令所需的 rs1 或 rs2 被前方尚未寫回暫存器、但已在管線後半段產出（EX/MEM 或 MEM/WB）的指令修改時，需在 ID Stage 直接截取該資料，避免 Stall。
2. Branch Hazard：當前方相依指令仍在 EX Stage 計算中（或是前方指令為 LW 讀取記憶體），由於資料尚未產生，Forwarding 無效，必須強制暫停 1 個時脈週期。

四、Exception 與中斷處理機制

1. Exception 偵測：本處理器針對以下三種符合 RISC-V 官方標準體系結構之例外，配置了偵測電路：

- **非法指令異常（Illegal Opcode Exception）：**當指令流控制單元解碼出未定義、不合法或未支援的指令時於 ID Stage 觸發，硬體寫入標準異常代碼 `mcause = 32'd2`。
- **軟體斷點（EBREAK Exception）：**用來中斷程式的運作，硬體寫入標準異常代碼 `mcause = 32'd3`。

- **環境呼叫異常 (ECALL Exception)**：用來執行需要更高權限才能執行的功能，硬體寫入標準異常代碼 `mcause = 32'd8`。

(註：有別於學長原始 MIPS 架構將整數算術溢位列為硬體異常之設計，本架構嚴格遵循 RISC-V 官方規範，算術溢位 (Overflow) 由軟體自行檢查。)

2. 機器模式控制狀態暫存器 (Machine-Mode CSRs)：為了實現例外現場的狀態保存，本專題設計了機器模式 (M-mode) 控制狀態暫存器，作為軟硬體協同的硬體基礎：

- **mepc (Machine Exception Program Counter)**：當例外發生時，硬體會將該故障指令的 PC 寫入 `mepc`，用作後續中斷處理程式執行完畢後返回原點。
- **mcause (Machine Cause Register)**：當例外發生時，硬體會根據 Exception 原因將對應故障編碼寫入此暫存器，供後續中斷處理程式進行故障排除。
- **mtvec (Machine Trap Vector Base Address)**：中斷向量入口基底位址暫存器，當例外發生時，會跳到該位址。本設計將初始值設為 `32'h0000_0080`。

3. Pipeline Flush： 一旦例外指令觸發，硬體會將後方指令全部沖刷，前方指令繼續執行，待例外指令流到 WB Stage 將例外發生位址和例外原因寫入 CSR。

貳、設計方法

一、 ID-Stage Early Branch Comparator

1.獨立分支比較器：將比較器與暫存器資料對接，根據 funct3 解碼，隨時進行 ($\geq$ 、 $<$ 、 $==$ 、 $\neq$) 比較，輸出 branch_take 信號。

```
case (funct3)
  3'b000: branch_take = (data1 == data2);           // BEQ
  3'b001: branch_take = (data1 != data2);           // BNE
  3'b100: branch_take = ($signed(data1) < $signed(data2)); // BLT
  3'b101: branch_take = ($signed(data1) >= $signed(data2)); // BGE
  3'b110: branch_take = (data1 < data2);           // BLTU
  3'b111: branch_take = (data1 >= data2);           // BGEU
default: branch_take = 1'b0;
endcase
```

2.計算跳躍位址：根據 branch_take 信號和 Control_unit 解碼出的分支信號進行跳躍位址的計算，輸出 PC_sel 信號進行位址選擇。

二、 Branch Forwarding Logic

1. 設計原理

由於分支判定提前至 ID Stage，若前方指令尚未寫回暫存器，將引發 RAW 資料危障。本模組會即時比對當前要使用的暫存器與後半段管線的暫存器編號，直接將最新數據拉回 ID Stage，消滅管線停頓。

與一般 Forwarding 架構類似，判斷時要先從 EXMEM (離 ID Stage

最近)開始，隨後才檢查 WB Stage，這樣才能確保拿到最新資料。除此之外還要檢查是否為 x0 暫存器，x0 暫存器恆為 0(不會被改寫)，如果將其 Forwarding 會使判斷出錯。

2.程式碼片段

```
always @(*) begin
    // ForwardC Logic (For Branch Rs1)
    ForwardC = 2'b00;
    if (EXMEM_RegWrite == 1'b1 && EXMEM_WriteReg != 5'd0 && EXMEM_WriteReg == ID_Rs)
        ForwardC = 2'b10;
    else if (WB_RegWrite == 1'b1 && WB_WriteReg != 5'd0 && WB_WriteReg == ID_Rs)
        ForwardC = 2'b01;

    // ForwardD Logic (For Branch Rs2)
    ForwardD = 2'b00;
    if (EXMEM_RegWrite == 1'b1 && EXMEM_WriteReg != 5'd0 && EXMEM_WriteReg == ID_Rt)
        ForwardD = 2'b10;
    else if (WB_RegWrite == 1'b1 && WB_WriteReg != 5'd0 && WB_WriteReg == ID_Rt)
        ForwardD = 2'b01;
end
```

三、 Branch Hazard

1.設計原理

當前方指令仍在 EX Stage 計算，或前方為 lw (Load) 指令且正在讀取記憶體時。由於資料尚未產生，無法依靠 Forwarding 解決數據衝突。BranchHazard 會強制 Pipeline 暫停一週期(若前方為 load 指令需要停止兩週期)，等待資料就緒。

2.程式碼片段

```

// JALR(Rs1)
if (PC_W_jr == 1'b1) begin
    if ((IDEX_RegWrite == 1'b1 && IDEX_Rd != 5'd0 && IDEX_Rd == ID_Rs) ||
        (EXMEM_MemRead == 1'b1 && EXMEM_WriteReg != 5'd0 && EXMEM_WriteReg == ID_Rs)) begin
        PC_Write_B = 1'b1;
        IFID_RegWrite_B = 1'b1;
        IDEX_Flush_B = 1'b1;
    end
end
// Branch(Rs1,Rs2)
else if (PC_W_Branch == 1'b1) begin
    if ((IDEX_RegWrite == 1'b1 && IDEX_Rd != 5'd0 && (IDEX_Rd == ID_Rs || IDEX_Rd == ID_Rt)) ||
        (EXMEM_MemRead == 1'b1 && EXMEM_WriteReg != 5'd0 && (EXMEM_WriteReg == ID_Rs || EXMEM_WriteReg == ID_Rt))) begin
        PC_Write_B = 1'b1;
        IFID_RegWrite_B = 1'b1;
        IDEX_Flush_B = 1'b1;
    end
end
end

```

四、CSR 實作與 Exception 處理

1.CSR 與 Zicsr 擴充指令實作：CSR 是 CPU 內的一種特殊暫存器，主要用於控制 CPU 行為並記錄當前狀態，本次我實作了三種 CSR 分別是 mtvec(中斷處理程式的記憶體入口位址)、mepc(Exception 或中斷發生位址)、mcause(中斷觸發原因)。與一般通用暫存器不同的是 CSR 因為關係到系統安全，所以無法被一般指令讀寫。在 RISC-V 中 Zicsr 指令就是專門用來讀寫 CSR 的指令集，他總共有六條 Atomic Read-Modify-Write 指令，確保讀寫過程不被打斷。

```

always @(posedge Clk or posedge Rst) begin
  if (Rst == 1'b1) begin
    // initial mtvec?avoid 0x0000_0000
    mtvec_out <= 32'h0000_0080;
    mepc_out <= 32'h0000_0000;
    mcause_out <= 32'h0000_0000;
  end
  else begin
    // when Exception save state
    if (Exception_W == 1'b1) begin
      mepc_out <= Trap_PC;
      mcause_out <= {28'b0, Trap_Cause};
    end
    // Software actively writes (extension functions) using CSR commands.
    else if (CSR_Write_Enable == 1'b1) begin
      case (CSR_Address)
        MIVEC_ADDR: mtvec_out <= (CSR_Op == 2'b10) ? (mtvec_out | CSR_Write_Data) :
                                (CSR_Op == 2'b11) ? (mtvec_out & ~CSR_Write_Data) : CSR_Write_Data;

        MEPC_ADDR: mepc_out <= (CSR_Op == 2'b10) ? (mepc_out | CSR_Write_Data) :
                                (CSR_Op == 2'b11) ? (mepc_out & ~CSR_Write_Data) : CSR_Write_Data;

        MCAUSE_ADDR: mcause_out <= (CSR_Op == 2'b10) ? (mcause_out | CSR_Write_Data) :
                                    (CSR_Op == 2'b11) ? (mcause_out & ~CSR_Write_Data) : CSR_Write_Data;
      endcase
    end
  end
end

```

2.Exception 處理：本架構使用 cause 模組來判斷 Exception 種類將對應的錯誤代碼寫入 CSR，並拉高對應 Stage 的 Flush 訊號。使用 EPC 模組將事故現場返回點（PC_EPC）寫入 CSR，使中斷處理程式處理完後能返回主程式繼續執行。

(1)cause 模組

```

else if (Illegal_OP_W == 1'b1) begin
  EXMEM_Flush_E = 1'b0;
  IDEX_Flush_E = 1'b1;
  IFID_Flush_E = 1'b1;
  PC_Exception_W = 1'b1;
  cause = 4'd2; //Illegal Instruction CODE: 2
end
else if (EBREAK_W == 1'b1) begin
  //EBREAK
  EXMEM_Flush_E = 1'b0;
  IDEX_Flush_E = 1'b1;
  IFID_Flush_E = 1'b1;
  PC_Exception_W = 1'b1;
  cause = 4'd3; //Breakpoint CODE: 3
end

```

(2)EPC 模組

```

always @(*) begin
    PC_EPC = 32'h0000_0000;
    // =====
    // For any "synchronization exception" (Illegal OP / EBREAK / ECALL),
    // the value written by mepc must always be the physical PC
    // =====
    if (Illegal_OP_W == 1'b1 || EBREAK_W == 1'b1 || ECALL_W == 1'b1) begin
        PC_EPC = IFID_PC_add4 - 32'h4;
    end
    // =====
    // Asynchronous interrupt: Pointer to the address of the next instruction to be executed.
    // =====
    else if (External_interrupt_W == 1'b1) begin
        PC_EPC = IFID_PC_add4;
    end
end
endmodule

```

3.Zicsr 指令

指令	描述
CSRRW/CSRRWI	將 CSR 的舊值讀入 rd，再將 rs1 的值/立即值寫入 CSR。
CSRRS/CSRRSI	將 CSR 的舊值讀入 rd，並將 rs1 / 立即值中為 1 的對應位元在 CSR 中設為 1。
CSRRC/CSRRCI	讀將 CSR 的舊值讀入 rd，並將 rs1 / 立即值中為 1 的對應位元在 CSR 中清零。

參、模擬結果

一、hazard.asm

1.組合語言

本程式主要在驗證基本資料路徑（Data Path）的穩定度，測試以下三項寫後讀（RAW）危障處理機制：

Forwarding：驗證連續算術運算時，EX/MEM 與 MEM/WB 的資料能否及時回傳給 ALU。

記憶體連續存取：第 5、6 行針對同一實體位址進行連續寫入（sw）與讀取（lw），驗證資料記憶體在連續週期切換下的控制穩定度。

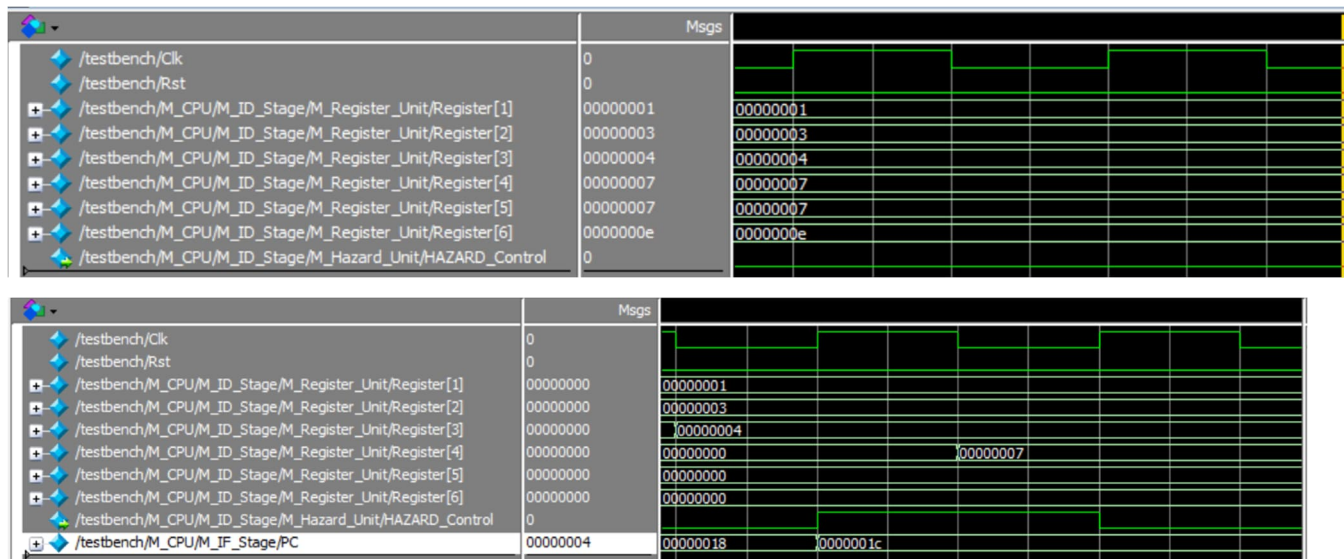
Load-Use Hazard：第 6、7 行故意觸發 Load-Use Hazard，驗證

Hazard_Unit.v 能否自動凍結管線、插入 1 週期 NOP，並隨後透過

Forwarding 繼續執行指令。

```
1      addi x1, x0, 1      # x1 = 1
2      addi x2, x1, 2      # x2 = x1 + 2 = 3 [Hazard]
3      addi x3, x0, 4      # x3 = 4
4      add  x4, x2, x3      # x4 = x2 + x3 = 3 + 4 = 7 [Hazard]
5      sw   x4, 0(x0)      # 把 7 寫入記憶體位址 0
6      lw   x5, 0(x0)      # x5 = 7
7      add  x6, x5, x5      # x6 = x5 + x5 = 7 + 7 = 14 [Hazard]
8      end:
9      jal  x0, end        # 無窮迴圈，讓波形停住
```

2.ModelSim 模擬



3.venus 模擬

	Registers	Memory
zero	0x00000000	
ra (x1)	0x00000001	
sp (x2)	0x00000003	
gp (x3)	0x00000004	
tp (x4)	0x00000007	
t0 (x5)	0x00000007	
t1 (x6)	0x0000000e	

二、branch.asm

1.組合語言

本程式專門用來驗證條件分支（bne、beq）與 Jump and Link Register

（jalr）指令提前至 ID Stage 時，衍生的 Data Hazard 處理機制：

Branch Forwarding: 第 5 至 7 行中, add 與 bne 之間隔了一條指令。

當 bne 進入 ID Stage 時, 最新數據位於 EX/MEM 管線暫存器中。

測試 Branch_Forwarding.v 能否直接將數值拉回 ID Stage 完成比較, 並跳轉至 target1 以避開下方的錯誤路徑。

Branch Hazard: 第 14、15 行中, add 與 beq 緊鄰相依。當 beq 進入 ID Stage 時, 前方指令仍在 EX Stage 運算。測試 Hazard_Unit.v 能否自動偵測衝突, 強制觸發 1 個時脈週期的 Stall, 待前方指令流動至 MEM 級後, 再透過 Forwarding 完成比較與跳轉。

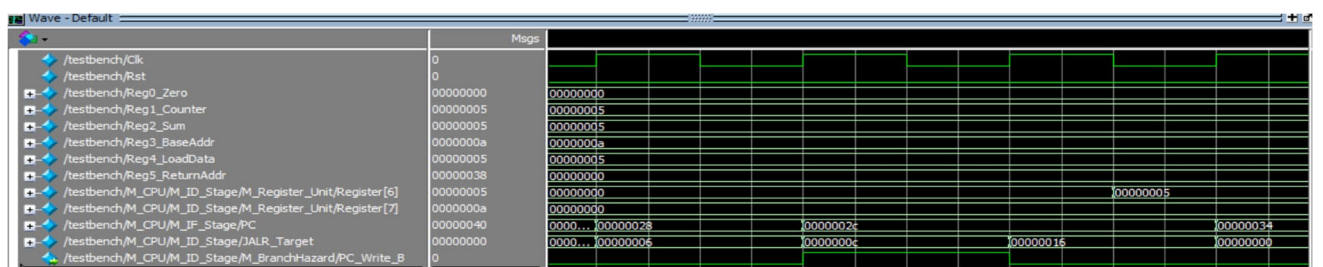
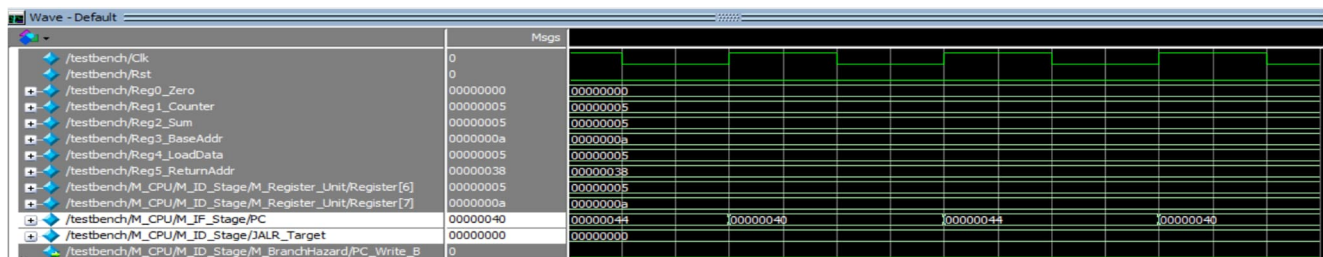
JALR 跳轉驗證: 第 24 行驗證 jalr 指令, 硬體需在 ID Stage 即時計算出跳轉目標位址 ($10 + 54 = 64$, 即 $0x40$), 使 PC 於下一週期跳轉至 target3, 同時將返回位址 ($PC+4 = 0x38$) 寫入通用暫存器 x5 中。

```

1  addi x1, x0, 5 # x1 = 5
2  addi x2, x0, 5 # x2 = 5
3  addi x3, x0, 10 # x3 = 10
4  # 測試 1: Branch Forwarding (EX/MEM 級前推, 完全不需 Stall)
5  add x4, x1, x0 # x4 = 5 + 0 = 5
6  addi x0, x0, 0
7  bne x4, x3, target1
8  # 應觸發 Branch_Forwarding
9  addi x5, x0, 1 # 錯誤路徑 (硬體出錯才會執行)
10 addi x5, x0, 2 # 錯誤路徑
11 target1:
12 # 測試 2: Branch Hazard (ID/EX 級衝突, 硬體必須強迫 Stall 1 週期)
13 addi x6, x0, 5 # x6 = 5
14 add x7, x6, x6 # x7 = 10
15 beq x7, x3, target2
16 # 應觸發 BranchHazard 產生 1 週期暫停 (PC_Write_B=1, IFID_RegWrite_B=1)
17 # 比較 10 == 10 條件成立, 跳躍到 target2
18 addi x5, x0, 4 # 錯誤路徑
19 addi x5, x0, 5 # 錯誤路徑
20 target2:
21 # 測試 3: JALR 指令跳轉與暫存器基底驗證
22 # 當前 x3 = 10。我們希望 JALR 精確跳躍到下方的 target3 (PC = 0x40)。
23 # 計算公式: JALR Target = (x3 + 立即值 54) = 10 + 54 = 64 = 0x40
24 jalr x5, 54(x3) # 執行 JALR, 同時將返回位址 PC+4 (0x38) 寫入 x5 暫存器
25 addi x5, x0, 7 # 錯誤路徑
26 addi x5, x0, 8 # 錯誤路徑
27 target3:
28 # [PC = 0x40]
29 end:
30 jal x0, end

```

2.ModelSim 模擬



3.venus 模擬

	Registers	Memory
zero	0x00000000	
ra (x1)	0x00000005	
sp (x2)	0x00000005	
gp (x3)	0x0000000a	
tp (x4)	0x00000005	
t0 (x5)	0x00000038	
t1 (x6)	0x00000005	
t2 (x7)	0x0000000a	

三、exception.asm

1.組合語言

本程式專門用來驗證 Zicsr 指令集的暫存器/立即值原子解碼通路，以及 Exception 的處理：

Zicsr 原子操作指令測試：第 8 至 14 行全面測試 Zicsr 擴充指令集。前三條針對 mepc 驗證暫存器型指令 (csrrw/csrrs/csrrc)；後三條針對 mcause 驗證立即值型指令 (csrrwi/csrrsi/csrrci)，確保通用暫存器讀取舊值、CSR 寫回新值的雙向原子通路完全正確。

CSR Hazard：第 15 至 17 行在 Exception 前方刻意插入 3 個 NOP 氣泡，確保前方的 csrrci 指令寫回後，後方的 Exception 才進場，避免控制狀態暫存器發生 RAW 資料危障。

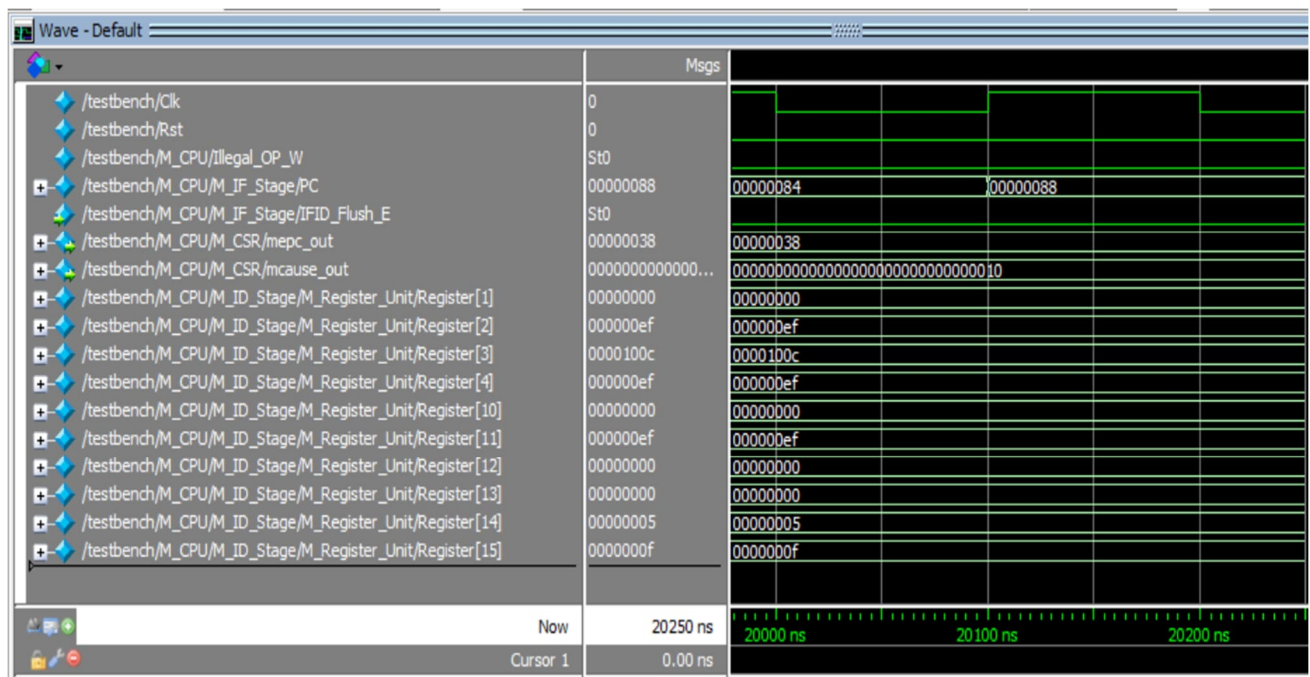
Exception 處理：第 19 行利用機器碼 0x0000007F 觸發 Illegal Opcode。驗證 Exception 偵測功能，之後將故障現場 PC 寫入 mepc_out (0x38)、錯誤原因寫入 mcause_out (2)，同時啟動管線沖刷(Flush)抹除後方無效指令，使 PC 跳躍至 mtvec 指定的 0x80 中斷處理程式。

```

1  addi x1, x0, 0          # x1 = 0
2  addi x2, x0, 239        # x2 = 239 (0x0000_00EF)
3  sw x2, 0(x1)            # 將 0xEF 寫入 [RAM 地址 0]
4  auipc x3, 1             # x3 = 0x000C + 0x1000 = 0x0000_100C
5  lbu x4, 0(x1)           # x4 = 0x0000_00EF (讀取 RAM 地址 0)
6
7  # ----- 6 條 Zicsr 全功能測試鍵 (刻意操作 mepc 與 mcause, 保留 mtvec 預設值為 0x80) -----
8  csrrw x10, 0x341, x2    # 【CSRRW】讀出 mepc 初始值 (0) 到 x10, 並把 x2 (0xEF) 寫入 mepc
9  csrrs x11, 0x341, x2    # 【CSRRS】讀出 mepc 舊值 (0xEF) 到 x11, 並與 x2 做 OR, mepc 仍為 0xEF
10 csrrc x12, 0x341, x4     # 【CSRRC】讀出 mepc 舊值 (0xEF) 到 x12, 並與 x4 (0xEF) 做 Clear, mepc 清零
11
12 csrrwi x13, 0x342, 5     # 【CSRRWI】讀出 mcause 舊值 (0) 到 x13, 並把立即值 5 直接寫入 mcause
13 csrrsi x14, 0x342, 10    # 【CSRRSI】讀出 mcause 舊值 (5) 到 x14, 並與立即值 10 (0x0A) 做 OR, mcause 變 15
14 csrrci x15, 0x342, 3     # 【CSRRCI】讀出 mcause 舊值 (15) 到 x15, 並與立即值 3 做 Clear, mcause 變 12 (0x0C)
15 NOP
16 NOP
17 NOP
18 # ----- 觸發特權異常爆炸點 -----
19 .word 0x0000007F        # 觸發 Illegal Opcode 異常!
20                          # 當前 PC = 0x0000_0038。
21                          # 系統自動鎖存: mepc_out = 0x38, mcause_out = 2。
22                          # 隨後, PC 強行跳躍到 mtvec 指定的入口: 32'h0000_0080。
23
24 jal x0, 0               # 常規防禦迴圈 (若無引發異常才會卡在這)
25
26 # ----- NOP Slide 軌道 (從位址 0x3C 一路補零延伸到 0x80) -----
27 0x3C ~ 0x7C : NOP      # 19 條硬體 Bubble/NOP 軌道, 用來填補空間至 0x80
28
29 # ----- 終點站: 中斷向量保護安全區 -----
30 0x84 : 0000006f -> jal x0, 0          # 【安全降落點】無限自圈迴圈!
31                                     # 當異常噴過來後, PC 會在 0x0000_0084和0x0000_0088之間循環。

```

2.ModelSim 模擬



四、CMDtest.asm

1.組合語言

本程式專門用來驗證剩餘未驗證指令的正確性，並驗證剩餘兩種

Exception (ebreak、ecall)。


```

# =====
# 1. U-Type & I-Type 邏輯與移位驗證 (共 10 條指令)
# =====
lui    x10, 0x12345    # x10 = 0x12345000 (LUI 驗證)
andi   x11, x10, 0x3F0 # x11 = 0 (ANDI 驗證)
ori    x12, x11, 10    # x12 = 10 (ORI 驗證)
xori   x13, x12, 15    # x13 = 10 ^ 15 = 5 (XORI 驗證)
slli   x14, x13, 2     # x14 = 5 << 2 = 20 (SLLI 驗證)
srli   x15, x14, 1     # x15 = 20 >> 1 = 10 (SRLI 驗證)
addi   x1, x0, -10     # x1 = -10 (補零普通指令, 做基底)
srai   x16, x1, 2      # x16 = -10 >>> 2 = -3 (0xFFFFFDD) (SRAI 驗證)
slti   x17, x1, 5      # x17 = (-10 < 5) ? 1 : 0 -> 預期為 1 (SLTI 驗證)
sltiu  x18, x1, 5      # x18 = (0xFFFFF6 < 5) 無正負號比較 -> 預期為 0 (SLTIU 驗證)

# =====
# 2. R-Type 算術、邏輯與移位驗證 (共 10 條指令)
# =====
addi   x2, x0, 15      # x2 = 15
sub     x19, x2, x13    # x19 = 15 - 5 = 10 (SUB 驗證)
sll     x20, x13, x14   # x20 = 5 << 20 = 5242880 (0x00500000) (SLL 驗證)
slt     x21, x1, x2     # x21 = (-10 < 15) ? 1 : 0 -> 預期為 1 (SLT 驗證)
sltu    x22, x1, x2     # x22 = (0xFFFFF6 < 15) 無正負號比較 -> 預期為 0 (SLTU 驗證)
xor     x23, x2, x13    # x23 = 15 ^ 5 = 10 (XOR 驗證)
srl     x24, x20, x14   # x24 = 5242880 >> 20 = 5 (SRL 驗證)
sra     x25, x1, x13    # x25 = -10 >>> 5 = -1 (0xFFFFFFF) (SRA 驗證)
or      x26, x2, x13    # x26 = 15 | 5 = 15 (OR 驗證)
and     x27, x2, x13    # x27 = 15 & 5 = 5 (AND 驗證)

# =====
# 3. Store & Load 尺寸與正負號擴展驗證 (共 5 條指令)
# =====
sh      x1, 0(x0)       # 將 -10 (0xFFFF6) 寫入 Data MEM[0] (SH 驗證)
lh      x4, 0(x0)       # 讀回半字組帶正負號擴展 -> 預期 x4 = 0xFFFFF6 (-10) (LH 驗證)
lhu     x5, 0(x0)       # 讀回半字組零擴展 -> 預期 x5 = 0x0000FFF6 (LHU 驗證)
sb      x1, 4(x0)       # 將 -10 (0xF6) 寫入 Data MEM[4] (SB 驗證)
lb      x6, 4(x0)       # 讀回位元組帶正負號擴展 -> 預期 x6 = 0xFFFFF6 (-10) (LB 驗證)

# =====
# 4. Branch 剩餘條件分支反向自我檢查 (共 4 條指令)
# =====
# 目前 x1 = -10 (0xFFFFF6), x2 = 15 (0x0000000F)
blt     x2, x1, error   # 15 < -10 為 False -> 預期不跳轉, 順向流過 (BLT 驗證)
bge     x1, x2, error   # -10 >= 15 為 False -> 預期不跳轉, 順向流過 (BGE 驗證)
bltu    x1, x2, error   # 0xFFFFF6 < 15 無正負號為 False -> 順向流過 (BLTU 驗證)
bgeu    x2, x1, error   # 15 >= 0xFFFFF6 無正負號為 False -> 順向流過 (BGEU 驗證)

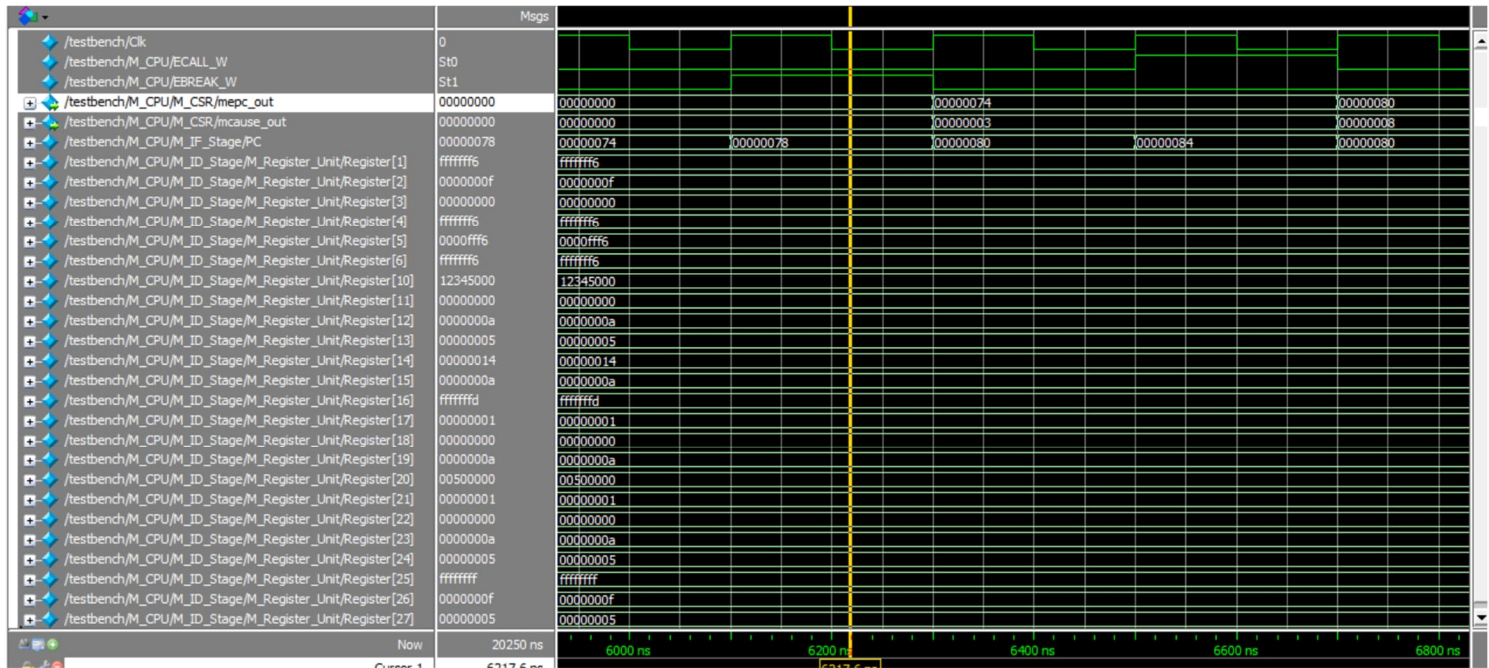
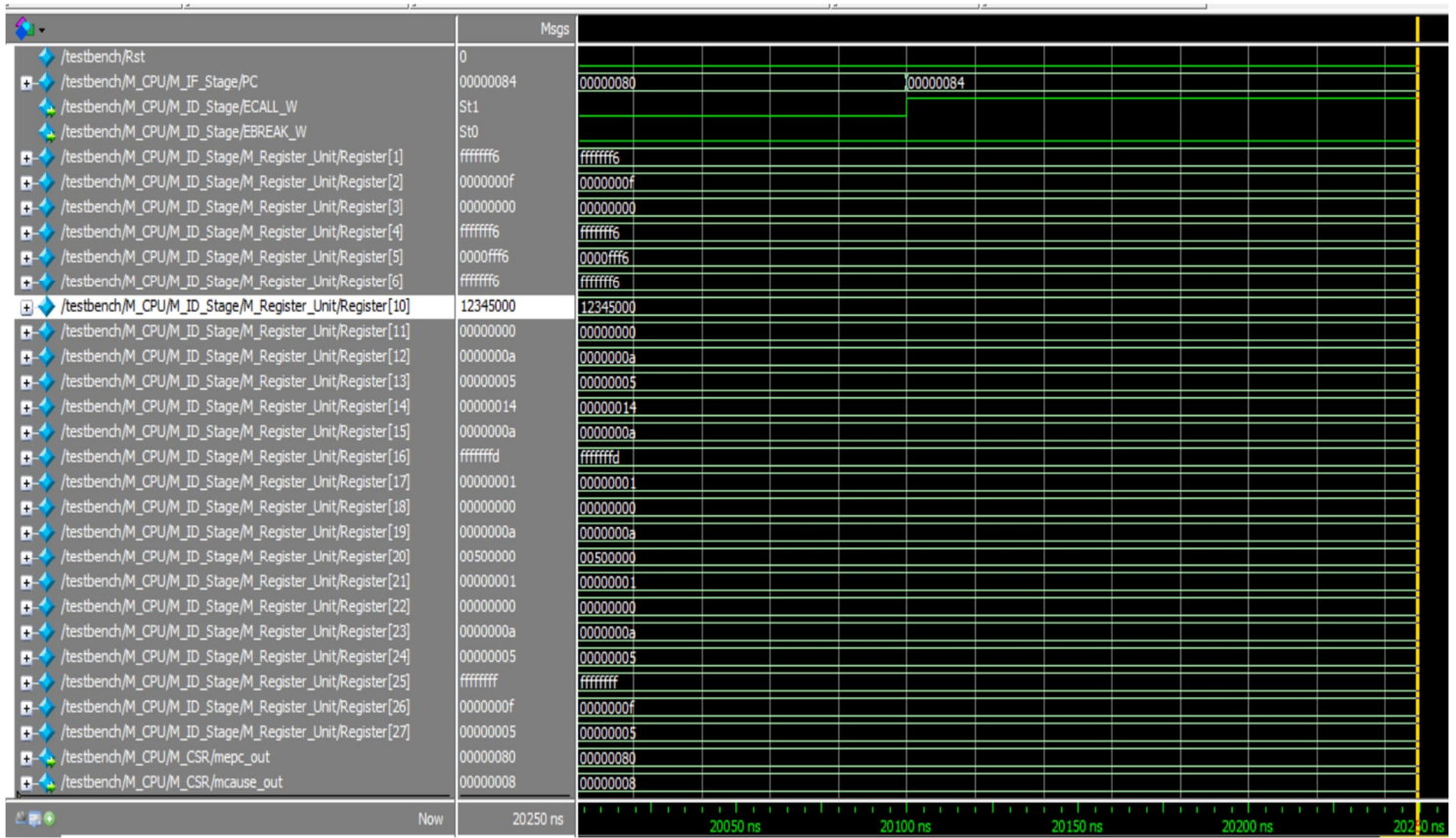
# =====
# 5. SYS 系統指令雙重例外爆發 (共 2 條指令)
# =====
ebreak                                     # 【第一波衝擊】執行到此, 硬體觸發 EBREAK 例外, 彈射至 0x80! (EBREAK 驗證)

# -----
# 錯誤攔截網 (若上面 Branch 算錯, PC 會被抓到這原地卡死, 不會進 0x80)
# -----
error:
    jal    x0, error
nop

# =====
# 中斷向量處理入口 (精確對齊 PC = 0x80, 第 33 行)
# =====
exception_handler:
    ecall                                     # 【第二波衝擊】降落到中斷區後立刻執行 ecall! (ECALL 驗證)
    jal    x0, exception_handler

```

2.ModelSim 模擬



肆、討論

這次將學長的 5 Stage Pipeline MIPS R2000 架構升級為具有 RV32I 基本指令與 Zicsr 擴充指令的 RISC-V 架構，並解決了學長遺留的 Branch Forwarding 問題。在實作過程中我發現 MIPS 和 RISC-V 不只在指令解碼上不同，在一些小細節上的處理也是(如：分支、記憶體端序、Exception 與中斷處理)，其中 Exception 的處理上我覺得最為複雜，MIPS 透過硬體內建的協處理器 (CP0)，在異常發生瞬間自動將返回位址 (EPC) 和原因 (Cause) 儲存在 CP0 內部的專用暫存器中。當中斷處理程式需要分析原因時，再透過 mfc0 指令將 CP0 的暫存器數值搬到通用暫存器中進行運算與處理(學長並未實現完整的 CP0，而是使用 Latch 進行簡化)；而 RISC-V 以一堆暫存器(CSR)取代了協處理器，省下了一堆中轉電路，並透過 Zicsr 指令直接對 CSR 進行操作，不需要將資料移到通用暫存器內再進行運算。

分支處理的部分我將分支判斷移到了 ID Stage 使分支懲罰降低，但帶來的是更為嚴重的 Data Hazard，因此 Branch_Forwarding 的實作是不可避免的，再加上 Hazard 的處理即可使分支判斷不會因為資料危障而出錯。

照理說只要有暫存器的讀寫就會有資料危障的問題，因此我實作

的 Zicsr 擴充指令也會有這個問題，叫做 CSR Hazard，在 exception.asm 的測試中我在 csrrci 指令後面引發 Exception，當兩邊都想寫入 CSR 暫存器，會導致最終結果出錯。目前我是在兩者間手動插入三個 NOP 讓 csrrci 指令執行完再引發 Exception，避免資料出錯。

因為 CSR Hazard 的處理相對複雜，且發生機率較低，通常會使用硬體來強制 Stall，目前因為時間有限先使用手動插入 NOP 來解決，未來可以嘗試將 Stall 機制實現。此外目前我的中斷處理程式仍停留在記錄問題，沒有進行處理，未來也需要將從中斷處理程式返回主程式的 mret 指令補齊。